

Kaizen Infinities — Unified CRM — Phase 1 Final Engineering Handoff

- KAIZEN INFINITIES — UNIFIED CRM
 - PHASE 1 — FINAL ENGINEERING HANDOFF
 - 1. BUILD SCOPE
 - 2. SYSTEM ARCHITECTURE
 - 3. DATA MODEL
 - 4. DATABASE SCHEMA
 - 5. API CONTRACT
 - 6. SCREEN / ROUTE SPECIFICATION
 - 7. COMPONENT SPECIFICATION
 - 8. RBAC
 - 9. ACTIVITY / EVENT / AUTOMATION MODEL
 - 10. INSTITUTION MAP + INSTITUTION 360
 - 11. MoU
 - 12. EDUCATION INTEGRATION
 - 13. SEARCH / FILES / NOTIFICATIONS
 - 14. INTEGRATIONS
 - 15. SECURITY / AUDIT / DATA GOVERNANCE
 - 16. DEPLOYMENT / ENVIRONMENTS
 - 17. IMPLEMENTATION ORDER
 - 18. DEVELOPER BACKLOG
 - 19. ACCEPTANCE TESTS
 - 20. PHASE 1 DEFINITION OF DONE
 - 21. UNRESOLVED DECISIONS

KAIZEN INFINITIES — UNIFIED CRM

PHASE 1 — FINAL ENGINEERING HANDOFF

Source of truth: the Phase 1 HTML wireframe (`Kaizen_CRM_Wireframe.dc.html` , 10 sections, round 1) and the Phase 1 Technical Bridge Documentation (`Tech_Spec_P1.pdf` , Parts I–XXII). This document does not redesign either — it converts them into one build-ready reference. Where the two sources leave a genuine gap, the gap is marked **Implementation Decision** with the minimum reasonable resolution, never a silent assumption.

1. BUILD SCOPE

BUILD (Phase 1 — this delivery)

Module	What ships
Foundation	Auth (login/logout/session/reset), Users, Roles/Permissions, App Shell + role-filtered Sidebar, audit-log write-through
Core CRM	Person (canonical, deduped), Organization (incl. Institution subtype), Relationship (polymorphic edge), Activity (polymorphic timeline), Task, Document upload, Global Search
Commercial CRM	Lead, Lead→Opportunity conversion, Opportunity + Kanban pipeline board, universal macro-stage pipeline
Institution Network	Institution Map (Tamil Nadu → District → Institution, default Madurai), geospatial filters, clustering, Institution Drawer, Institution 360, Institution Activity Timeline
MoU	MoU as a first-class record with full lifecycle (Proposed → Negotiating → Approved → Signed → Active → Expiring → Expired/Renewed), expiry automation, renewal workflow
Education (CRM-integrated only)	Admissions pipeline, Course recommendation engine (rule-based, not AI), Student 360 (layered on Person), Enrollment, Attendance, Daily Progress, role-based Learning Dashboards (Student/Trainer/Admin)
Executive Dashboard	Revenue, pipeline, win rate, collections, active-MoU KPIs; revenue-by-vertical; critical tasks; recent activity feed — role-scoped visibility
Notifications	In-app + email for lead assignment/untouched, task due/overdue, opportunity stage change, MoU expiring, student-at-risk, institution-inactive, payment-pending, proposal-pending
Cross-cutting	RBAC enforcement, audit logging, dedup-on-create, permission-scoped document storage

DEFER (explicitly scoped to Phase 2, referenced by Phase 1 data model but not built)

Item	Why deferred
Campaigns	Referenced by Lead.source but no Campaign CRUD/UI in Phase 1
Projects (full delivery/Gantt/resourcing)	Project entity exists as a reference target for won Opportunities; only status/milestone tracking, not full PM tooling
Standalone Communications/Activities hub UI polish, richer Task views	Activities top-nav inbox exists at MVP fidelity; deeper triage UX is Phase 2
Certifications, Placements, Alumni screens	Entities and relationships exist in the data model (Part III/XIII); dedicated UI ships Phase 2
Vertical-specific pipeline views (SAP, Cybersecurity, AI/Automation, Partnerships, Research)	Universal macro-stage pipeline ships Phase 1; per-vertical micro-stage UI is Phase 2
Reports & Analytics suite (geographic intelligence, customer health, workforce demand)	Executive Dashboard KPIs ship Phase 1; the broader reporting suite is Phase 3
Settings & role/permission management UI	RBAC is enforced Phase 1; a self-service admin UI to edit roles/permissions is Phase 2/3
Tablet/mobile responsive polish	Responsive behavior notes exist per screen (collapse/overlay rules); full tablet/mobile wireframes are a later round
AI scoring/recommendation models, “AI Insight” cards with real inference	Phase 1 captures the data these need (Part XVI); no model ships — any “AI Insight” UI element in Phase 1 is either omitted or clearly labeled as a rule-based placeholder, never a live model
Fuzzy/phonetic person deduplication	Phase 1 ships exact phone/email dedup only

EXTERNAL / INTEGRATED (Phase 1 depends on these but does not own them)

System	Phase 1 role
WhatsApp Business API	Inbound/outbound activity logging
Telephony / click-to-call	Call activity logging
Email (SMTP/provider API)	Communication logging
Payment gateway / UPI	Payment confirmation → Payment record
Website forms	Inbound Lead creation
Geocoding provider	Institution lat/Ing resolution at creation
MongoDB Atlas (incl. Search, geospatial, Scheduled Triggers)	Primary datastore, search, geospatial, background jobs

Object storage (S3-compatible)	Document/file binaries
--------------------------------	------------------------

OUT OF SCOPE (Phase 1 — not built, not integrated, not simulated)

Item	Note
Video hosting, lesson authoring, quiz engine, assignment engine, question banks, SCORM, full LMS content management, advanced LMS content analytics	Explicitly outside the CRM + Education Management boundary — Education in Phase 1 is limited to Students, Courses, Career Paths, Recommendations, Enrollments, Attendance, Daily/Learning Progress, Certification, Placement, Alumni as CRM-integrated capabilities, not a conventional LMS
LMS integration, HRMS integration, ERP/Accounting integration, Calendar sync, Marketing-system integration	Real and specified (Part 14) but sequenced Phase 2/Future — no Phase 1 workflow depends on them
Multi-tenancy	Kaizen is single-tenant in Phase 1; no <code>tenant_id</code> partitioning is built
Physical graph database	Relationships are a logical graph over MongoDB aggregation, never a separate graph engine (see §2)
Dedicated data warehouse	A daily aggregation collection covers Phase 1 analytics; no ETL/warehouse is built
MFA	Not included unless the organization mandates it before build start (open decision, §21)

2. SYSTEM ARCHITECTURE

One recommended architecture — no hypotheticals.

Layer	Technology	Phase 1 responsibility
Client	React SPA (existing frontend stack)	Renders all Phase 1 screens; holds URL/local UI state; calls the API layer only — no direct DB access
API layer	Node.js REST API (existing backend stack)	Auth, validation, business rules, permission checks; orchestrates DB + search + cache + storage
Primary datastore	MongoDB (Atlas)	System of record for every entity — people, organizations, institutions, relationships, activities, leads, opportunities, MoUs, education records
Search	MongoDB Atlas Search (Lucene-based, same cluster)	Full-text/fuzzy search across people, organizations, leads, opportunities — no separate search cluster
Geospatial	MongoDB native 2dsphere index	Institution Map viewport, radius, and district queries
Cache	Redis (or Atlas in-memory equivalent)	District/institution counts, map cluster precomputation, session tokens, rate limiting
File storage	Object storage (S3-compatible), metadata in MongoDB	MoU documents, contracts, attachments — binaries are never stored in MongoDB documents
Background jobs	Queue/worker (e.g. BullMQ on Redis, or Atlas Scheduled Triggers)	MoU expiry checks, notification dispatch, daily analytics aggregation, map cluster precomputation
Notifications	In-app store (<code>notifications</code> collection) + email provider	Event-driven alerts; email reuses the same event payload as in-app
External integrations	WhatsApp/telephony/email APIs, payment gateway, geocoding provider, website-form webhooks	See §14

The operational database question — resolved

MongoDB is retained as the single system of record. “Relationship” is implemented as a **logical relationship graph** — a first-class `relationships` collection traversed via aggregation (`$lookup` / `$graphLookup`) — **not a physical graph database**. This is a firm decision, not a placeholder:

- The existing Education CRM already runs on MongoDB in production; migrating to PostgreSQL would require a parallel schema-design effort, a data migration, and a change in developer tooling for no Phase 1 capability the CRM actually needs.
- The document model fits the mixed CRM + Education domain (variable fields per institution/education record) without excess joins.
- Native geospatial (2dsphere) and search (Atlas Search) cover Phase 1 needs on the same cluster, without standing up Elasticsearch or a separate geo engine.
- Relationship depth in Phase 1 (person→org→institution, 1-3 hops) is fully answered by `$graphLookup` ; a dedicated graph database (e.g. Neo4j) would add a second operational system to keep in sync for a traversal need that doesn’t exist yet.

Option considered	Verdict
MongoDB (recommended)	Adopt / continue
PostgreSQL	Not recommended for Phase 1 — reconsider only if transactional/financial integrity requirements grow substantially
PostgreSQL + pgvector	Not recommended for Phase 1 — solves an AI/embedding-search need the product doesn’t have yet; revisit only if a semantic-search/RAG feature is scoped
Relational + graph hybrid	Not recommended — reserve for a future feature needing deep multi-hop traversal or graph algorithms, not a Phase 1 default

Implementation decision: No migration is required for Phase 1; continue on the existing MongoDB cluster/Atlas project. If write volume on `activities` / `audit_logs` grows enough to require sharding, shard key candidates are `owner_id` (or `tenant_id` if multi-tenancy is ever introduced — not required for Phase 1).

Authentication & Maps/geospatial

- **Authentication:** session-token based (JWT or opaque + server-side revocation), email+password, 8-hour sliding session with refresh, forced re-login after 30 days idle. See §8 for the full auth/RBAC specification.
- **Maps/geospatial:** MongoDB 2dsphere index on `organizations.institution_details.location` , queried via `$geoWithin` / `$nearSphere` , combined with non-geo filters (district, type, management_type) in a single query — never fetched then filtered client-side. See §10.

3. DATA MODEL

Every entity carries: `_id` , `created_at` , `created_by` , `updated_at` , `updated_by` , `source` (`manual` | `import` | `integration` | `conversion`). Soft-delete via `deleted_at` ; hard deletes are reserved for governance-mandated erasure (§15).

Universal CRM entities

Entity	Purpose	Primary key	Key relationships	Owner field	Lifecycle
Person	Canonical human record — one row per real human regardless of how many roles they hold	<code>_id</code>	→ Relationship → Organization/Institution; → Activity; → Lead; → Opportunity	<code>owner_id</code>	active → merged (dedup) — never hard-deleted
Organization	Any company, government body, vendor, or partner Kaizen deals with. Institution is a subtype (see below)	<code>_id</code>	→ Institution (subtype); → Relationship; → Opportunity; → MoU	<code>owner_id</code>	active, hierarchy via <code>parent_org_id</code>
Relationship	First-class, polymorphic edge between any two entities, with type, role, status, and time bounds	<code>_id</code>	<code>from_type/from_id ↔ to_type/to_id</code> (polymorphic)	<code>owner_id</code>	active → inactive/ended; new role = new Relationship record, never an overwrite

Lead	Early-stage interest record, pre-qualification	<code>_id</code>	→ Person; → Organization/Institution; → converts to → Opportunity	<code>owner_id</code>	created → converted (kept, not deleted, as pipeline-entry audit trail)
Opportunity	Qualified, working deal	<code>_id</code>	→ Account (Organization/Institution); → Contact (Person); → Activity; → MoU; → Project	<code>owner_id</code>	Discovered → ... → Won/Lost → Delivering → Outcome → Renew/Expand/Refer (see \$5 stage table)
Activity	Universal interaction record (call, meeting, visit, etc.)	<code>_id</code>	polymorphic <code>related_*</code> references to Person/Organization/Institution/Lead/Opportunity/Project	<code>actor_id</code>	append-only interaction log
Task	Actionable follow-up, may originate from an Activity	<code>_id</code>	→ related entity; → assignee	<code>assignee_id</code>	open → complete/reassigned
Communication	Logged message content (email/WhatsApp thread), distinct from the Activity summarizing it	<code>_id</code>	→ Activity; → Person	<code>actor_id</code>	append-only
Document	File metadata attached to any entity; binary lives in object storage	<code>_id</code>	polymorphic <code>attached_to</code>	<code>uploaded_by</code>	versioned (new record per re-upload, shared <code>document_group_id</code>)
Note	Free-text note attached to any entity (lighter-weight than Activity)	<code>_id</code>	polymorphic <code>attached_to</code>	<code>author_id</code>	—
Tag	Free-form label attachable to any entity for filtering	<code>_id</code>	polymorphic <code>attached_to</code>	—	—
MoU	Formal agreement with an Organization/Institution	<code>_id</code>	→ Organization/Institution; → Document; → Activity; → Opportunity	<code>owner_id</code>	Proposed → Negotiating → Approved → Signed → Active → Expiring → Expired/Renewed
Campaign (<i>Phase 2, referenced only</i>)	Marketing/outreach initiative that generates Leads	<code>_id</code>	→ Lead	<code>owner_id</code>	—
Project (<i>reference target only in Phase 1</i>)	Delivery-phase work after an Opportunity is won	<code>_id</code>	→ Opportunity; → Activity; → Payment	<code>owner_id</code>	—
Payment	Financial transaction tied to an Opportunity/Project/Enrollment	<code>_id</code>	→ Opportunity/Project/Enrollment	<code>owner_id</code>	pending → received/reconciled

Education entities (CRM-integrated, not an LMS)

Entity	Purpose	Key relationships
Course	Catalog offering (curriculum, duration, fees)	→ Enrollment; → CareerPath
CareerPath	Grouping of courses toward an outcome (job role)	→ Course
Student (<code>StudentProfile</code>)	Education-context profile layered 1:1 on a Person, not a separate identity	1:1 → Person; → Enrollment
Enrollment	A Student’s registration in a Course/batch	→ Student; → Course; → Payment; → Attendance
Attendance	Per-session attendance record	→ Enrollment (referenced, not embedded)
DailyProgress	Per-day academic progress log	→ Enrollment (referenced, not embedded)
Recommendation	Rule-based suggested next course/action for a Student	→ Student; → Course

Institution entities

Institution is **not** a parallel entity — it is the `organizations` collection with `category = institution` and an `institution_details` sub-document (geospatial location, `institution_type` , `management_type` , academic identifiers). Certification, Placement, and Alumni status are expressed as Relationship records (`studied_at` ended + `certified_in / employed_by` added) plus Enrollment outcome fields, not separate parallel person tables — see §12.

Commercial entities

Lead, Opportunity, MoU, Project, Payment (see Universal table above) — grouped here conceptually as the revenue-facing subset of the universal model.

System entities

Entity	Purpose	Key relationships
User	Login identity for a Kaizen employee (distinct from Person, which represents anyone, including non-employees)	→ Role; may reference Person if the employee is also tracked as a contact
Role	Named permission set (Founder, Admin, Sales, etc.)	→ Permission
Permission	Resource + action grant	→ Role
AuditLog	Immutable, append-only change record	polymorphic subject reference
Notification	In-app/email alert record	polymorphic subject reference; → recipient (User)

Explicit non-duplication rule

No redundant entities exist. A single **Person** can simultaneously be Student → Alumni → Employee → Corporate Contact → Decision Maker — this is the ordered set of that Person’s time-bounded Relationship records, never a new Person row. A single **Organization** can simultaneously be School → MoU Partner → Training Client → Employer — this is the same Organization row carrying multiple `category / relationship` states, never a new Organization row. This rule is enforced structurally (dedup-on-create, §4) and validated by the critical acceptance test in §19.

4. DATABASE SCHEMA

MongoDB collections, one per canonical entity group. Embed only tightly-coupled, unbounded-append child data that is never queried independently; reference everything else by `_id` .

users

Field	Type	Required	Default	Index	Unique	Reference	Description
<code>_id</code>	ObjectId	yes	—	PK	yes	—	Canonical identifier
<code>email</code>	string	yes	—	yes	yes	—	Login identifier
<code>password_hash</code>	string	yes	—	no	no	—	bcrypt/argon2
<code>role_id</code>	ObjectId	yes	—	no	no	→ roles	Permission set
<code>person_id</code>	ObjectId	no	null	no	no	→ persons	If employee is also a CRM contact
<code>status</code>	enum	yes	<code>active</code>	no	no	—	<code>active \</code> <code>inactive</code>
<code>last_login_at</code>	datetime	no	null	no	no	—	—

roles / embedded permissions[]

Field	Type	Required	Default	Notes
<code>_id</code>	ObjectId	yes	—	Canonical identifier
<code>name</code>	string	yes	—	Founder, Admin, Sales, Telecaller, Education Counsellor, Trainer, Project Manager, Workforce/Placement, Finance, Marketing
<code>permissions[]</code>	array of <code>{resource, action}</code>	yes	—	Embedded — small, static, edited together with the role

persons

Field	Type	Required	Default	Index	Unique	Description
<code>_id</code>	ObjectId	yes	—	PK	yes	Canonical identifier
<code>full_name</code>	string	yes	—	Atlas Search text	no	—
<code>primary_phone</code>	string	yes*	—	sparse B-tree	unique-ish	*At least one of phone/email required at creation for dedup
<code>primary_email</code>	string	yes*	—	sparse B-tree	unique-ish	*See above
<code>additional_phones[]</code> / <code>additional_emails[]</code>	array	no	<code>[]</code>	no	no	Captured over time from different activities/sources; embedded (small, bounded)
<code>current_location</code>	<code>{city, district, state}</code>	no	null	no	no	—
<code>social_identifiers</code>	<code>{linkedin, whatsapp}</code>	no	null	no	no	Only where a workflow actually uses it
<code>external_ids[]</code>	array< <code>{system, id}</code> >	no	<code>[]</code>	B-tree on <code>.id</code>	no	Links to source systems (education LMS ID, import batch ID)
<code>dedupe_status</code>	enum	yes	<code>active</code>	no	no	<code>active \</code> <code>merged \</code> <code>flagged_duplicate</code>
<code>merged_into</code>	ObjectId	no	null	no	no	Set when this record was merged into a surviving Person

organizations

Field	Type	Required	Default	Index	Unique	Description
<code>_id</code>	ObjectId	yes	—	PK	yes	Canonical identifier
<code>name</code>	string	yes	—	Atlas Search text	no	—
<code>category</code>	enum	yes	—	compound w/ district etc.	no	<code>company \</code> <code>institution \</code> <code>government \</code> <code>vendor \</code> <code>partner \</code> <code>other</code> — drives which sub-document is populated
<code>industry</code>	string	no	null	no	no	—
<code>parent_org_id</code>	ObjectId	no	null	no	no	Self-reference; supports Parent → Subsidiary → Branch → Department hierarchy (Department = Organization with a parent)
<code>contacts[]</code>	array< <code>{person_id, role_label}</code> >	no	<code>[]</code>	no	no	Denormalized quick-reference cache; authoritative link is the Relationship record, not this array
<code>institution_details.institution_type</code>	enum	if <code>category=institution</code>	null	compound B-tree	no	<code>school \</code> <code>college \</code> <code>university \</code> <code>training_institution \</code> <code>other</code>
<code>institution_details.management_type</code>	enum	if <code>category=institution</code>	null	compound B-tree	no	<code>government \</code> <code>private \</code> <code>aided \</code> <code>autonomous</code>
<code>institution_details.location</code>	GeoJSON Point <code>{lat, lng}</code>	if <code>category=institution</code>	null	2dsphere	no	Backs the Institution Map
<code>institution_details.address</code>	string	if <code>category=institution</code>	null	no	no	—
<code>institution_details.district</code>	string	if <code>category=institution</code>	null	compound B-tree	no	Defaults filter to Madurai
<code>institution_details.taluk</code>	string	no	null	no	no	Where available
<code>institution_details.external_identifier</code>	string	no	null	no	no	Code from an authoritative education directory (e.g. AISHE/UDISE), never fabricated — see §10

relationships

Field	Type	Required	Default	Index	Description
<code>_id</code>	ObjectId	yes	—	PK	Canonical identifier
<code>from_type</code> / <code>from_id</code>	enum / ObjectId	yes	—	compound B-tree	Polymorphic (Person, Organization, Institution)
<code>to_type</code> / <code>to_id</code>	enum / ObjectId	yes	—	compound B-tree	Polymorphic
<code>relationship_type</code>	enum	yes	—	B-tree	<code>studied_at \</code> <code>employed_by \</code> <code>decision_maker_for \</code> <code>partner_of \</code> <code>referred \</code> <code>alumnus_of \</code> <code>trainer_at \</code> <code>vendor_of \</code> <code>other</code>
<code>role</code>	string/enum	no	null	no	Free-text/enum label for role-bearing org links (e.g. Placement Officer, Decision Maker)
<code>status</code>	enum	yes	<code>active</code>	B-tree (compound w/ type)	<code>active \</code> <code>inactive \</code> <code>ended</code>
<code>strength</code>	enum	no	derived	no	<code>weak \</code> <code>moderate \</code> <code>strong</code> — manually set or derived from activity recency/frequency
<code>owner_id</code>	ObjectId	yes	—	no	CRM user accountable for the relationship

start_date / end_date	date	yes / no	— / null	no	Nullable end_date = ongoing
-----------------------	------	----------	----------	----	-----------------------------

Leads

Field	Type	Required	Default	Index	Description
_id	ObjectId	yes	—	PK	—
person_id	ObjectId	yes	—	no	Always points at a Person (created if it doesn't exist yet)
organization_id / institution_id	ObjectId	no	null	no	Nullable
source	enum	yes	—	no	website_form \\\ referral \\\ campaign \\\ walk_in \\\ event \\\ cold_outreach \\\ other
vertical , product	string	yes	—	no	Which Kaizen offering
owner_id	ObjectId	yes	—	no	—
stage , status	enum	yes	new	compound B-tree (stage, owner_id)	Per universal pipeline (\$5)
score	number 0–100	yes	0	no	Transparent rule-based score (recency + activity count + source weight) — not a model
last_activity_at , next_action	datetime/string	no	null	no	Drives “lead untouched” notification (\$9)
converted_to_opportunity_id	ObjectId	no	null	no	Set on conversion

opportunities

Field	Type	Required	Default	Index	Description
_id	ObjectId	yes	—	PK	—
account_type / account_id	enum/ObjectId	yes	—	no	Organization or Institution
contact_id	ObjectId	yes	—	no	→ Person
vertical , product , owner_id	string/ObjectId	yes	—	no	—
stage	enum	yes	discovered	compound B-tree (stage, owner_id, expected_close_date)	Universal macro-stage (\$5)
vertical_stage	string	no	null	no	Vertical-specific micro-stage label
probability	number	yes	stage default	no	Defaults from stage lookup, editable per deal
value , currency	number/string	yes	—	no	—
expected_close_date , source	date/string	no	null	no	—
related_activities[]	array	no	[]	no	References, not embeds — activities remain source of truth
proposal_doc_id , contract_doc_id	ObjectId	no	null	no	→ documents
outcome	enum	no	null	no	won_delivered \\\ won_cancelled \\\ lost_reason enum

activities

Field	Type	Required	Default	Index	Description
_id	ObjectId	yes	—	PK	—
type	enum	yes	—	no	call \\\ whatsapp \\\ email \\\ meeting \\\ visit \\\ workshop \\\ training \\\ event \\\ follow_up \\\ proposal \\\ payment \\\ mou_activity \\\ research \\\ survey \\\ other
actor_id	ObjectId	yes	—	no	User who logged it
participants[]	array	no	[]	no	—
related_person_id / related_organization_id / related_institution_id / related_lead_id / related_opportunity_id / related_project_id	ObjectId	no (≥0 set)	null	B-tree, one per field	Polymorphic timeline source — a single Activity satisfies every related entity’s timeline query from one write
timestamp , duration , outcome , notes	mixed	yes/no	—	timestamp descending B-tree	—
attachments[]	array	no	[]	no	→ documents
next_action	string	no	null	no	Drives auto Task creation
owner_id	ObjectId	yes	—	no	—

tasks

Field	Type	Required	Default	Index	Description
_id	ObjectId	yes	—	PK	—
related_activity_id	ObjectId	no	null	no	→ activities
related_entity.type / related_entity_id	enum/ObjectId	no	null	no	Polymorphic
assignee_id	ObjectId	yes	—	no	—
due_date	date	yes	—	no	—
status	enum	yes	open	no	open \\\ complete \\\ overdue
priority	enum	yes	normal	no	low \\\ normal \\\ high

mous

Field	Type	Required	Default	Index	Description
_id	ObjectId	yes	—	PK	—
organization_id /	ObjectId	at least one	—	no	—

institution_id					
scope	string	yes	—	no	Free text/structured scope
vertical , owner_id	string/ObjectId	yes	—	no	—
document_id	ObjectId	no	null	no	→ documents (signed file)
start_date / end_date	date	yes	—	compound B-tree (end_date , status)	Drives expiry job
status	enum	yes	proposed	—	proposed \\\ negotiating \\ approved \\\ signed \\\ active \\ expiring \\\ expired \\\ renewed
renewed_from_id	ObjectId	no	null	no	Points at the MoU this one supersedes

documents

Field	Type	Required	Default	Index	Description
_id	ObjectId	yes	—	PK	—
filename , storage_key , mime_type , size	mixed	yes	—	no	Binary lives in object storage; storage_key is the pointer
attached_to_type / attached_to_id	enum/ObjectId	yes	—	no	Polymorphic
version , document_group_id	number/ObjectId	yes	1	no	New record per re-upload, shared group id, prior versions retrievable
uploaded_by	ObjectId	yes	—	no	—

notes / tags

Field	Type	Required	Index	Description
body / label	string	yes	no	—
attached_to_type / attached_to_id	enum/ObjectId	yes	no	Polymorphic
author_id	ObjectId	yes (notes only)	no	—

courses / career_paths

Field	Type	Required	Index	Description
name , duration , fees	mixed	yes	no	—
curriculum_ref	string	no	no	Reference only — no curriculum authoring in Phase 1
career_path_id	ObjectId	no	no	→ career_paths

students

Field	Type	Required	Default	Index	Description
_id	ObjectId	yes	—	PK	—
person_id	ObjectId	yes	—	unique	1:1 → persons
academic_details	object	no	{}	no	Interests, qualification, career goal — feeds the recommendation engine

enrollments

Field	Type	Required	Default	Index	Description
student_id , course_id	ObjectId	yes	—	no	—
batch	string	no	null	no	—
status	enum	yes	enrolled	no	Tracks admissions macro-stage mapping
enrolled_at	date	yes	—	no	—

attendance

Field	Type	Required	Index	Description
enrollment_id	ObjectId	yes	B-tree	Referenced, NOT embedded in enrollment — unbounded per-enrollment growth would blow past MongoDB's practical per-document size limits and make the enrollment document expensive to load for a field almost never needed with it
session_date , status	date/enum	yes	no	—

daily_progress

Field	Type	Required	Index	Description
enrollment_id	ObjectId	yes	B-tree	Referenced, same reasoning as attendance
date , notes , score	mixed	yes/no	no	—

recommendations

Field	Type	Required	Index	Description
student_id , course_id	ObjectId	yes	no	—
reason , status	string/enum	no	no	Rule-based weighted-factor output (§12), not a model

projects / payments

Field	Type	Required	Index	Description
opportunity_id / project_id	ObjectId	yes	no	—

amount , status , due_date	mixed	yes	no	—
----------------------------	-------	-----	----	---

communications

Field	Type	Required	Index	Description
_id	ObjectId	yes	PK	—
activity_id	ObjectId	yes	B-tree	→ activities — the Activity summarizing this thread
person_id	ObjectId	yes	B-tree	→ persons
channel	enum	yes	no	email \ whatsapp
body , direction , sent_at	mixed	yes	no	Raw logged message content, distinct from the Activity's summary/outcome

Certification, Placement, Alumni — explicitly not separate collections

Per the no-duplicate-identity architectural principle (§3), **Certification**, **Placement**, and **Alumni** are not modeled as parallel entities/collections. They are expressed entirely through the existing machinery:

- **Certification** — an `activities` record (`type` capturing certificate issuance, e.g. `training` outcome) plus, where a formal credential is tracked, a `certified_in`-style `relationships` record from the Person to the certifying Institution/Organization. No new collection.
- **Placement** — a new `employed_by` `relationships` record from the Student's Person to the hiring Organization, created alongside an `opportunities` / `activities` record for the placement itself. No new collection.
- **Alumni** — the same Person's existing `studied_at` `relationships` record gets an `end_date` ; no new record, no new collection.

Introducing dedicated `certifications` / `placements` / `alumni` collections would duplicate what Relationship + Activity + Enrollment outcome fields already model, and was explicitly avoided per the “do not create redundant entities” build constraint. Full behavior is specified in §12.

notifications

Field	Type	Required	Index	Description
recipient_id	ObjectId	yes	B-tree	→ users
type , payload	string/object	yes	no	Mirrors the triggering event
read_at	datetime	no	no	null = unread

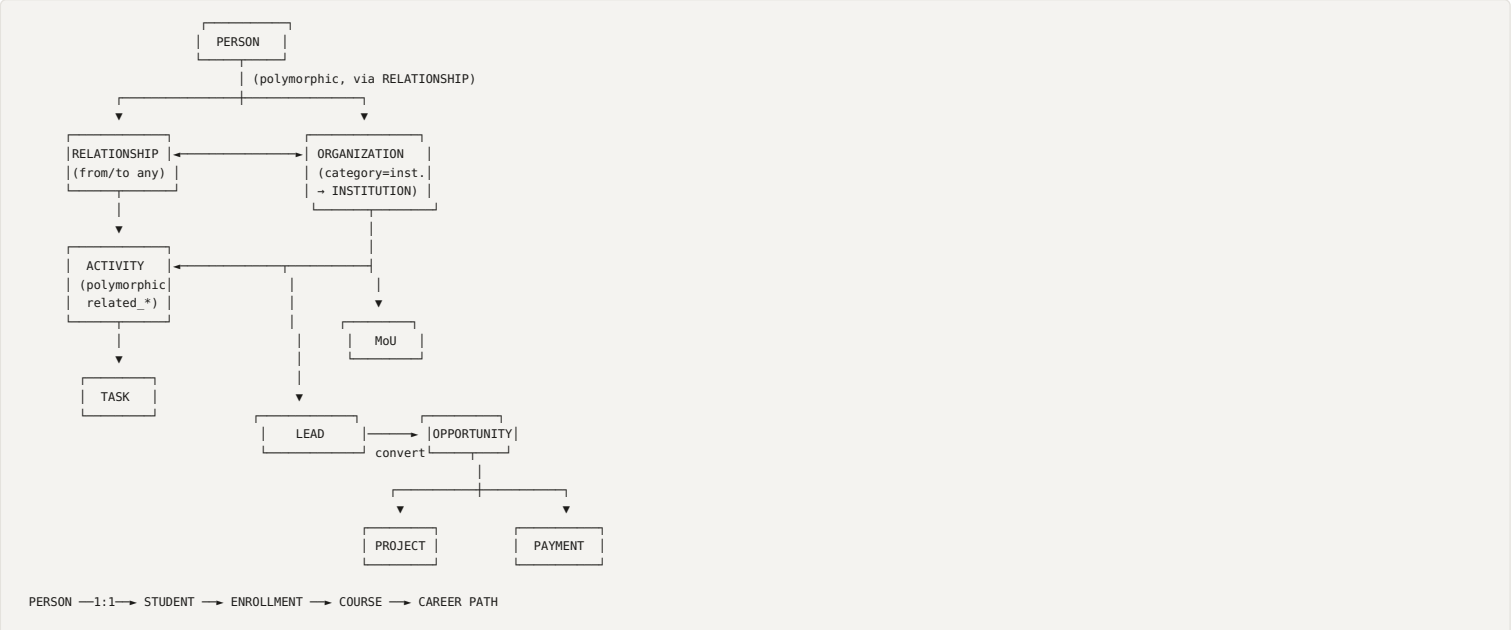
audit_logs

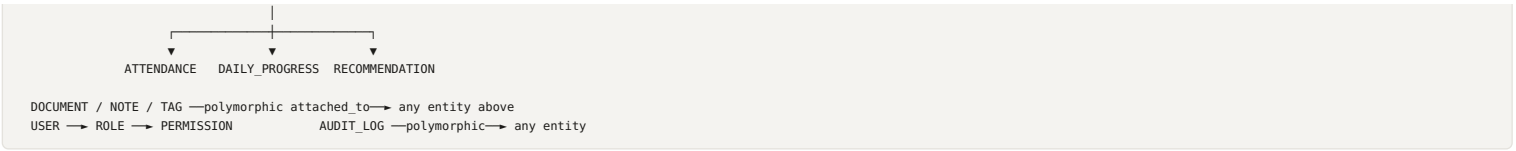
Field	Type	Required	Index	Description
subject_type , subject_id	enum/ObjectId	yes	no	Polymorphic subject reference
actor_id , action	ObjectId/enum	yes	no	<code>create</code> \ <code>update</code> \ <code>delete</code>
before{} , after{}	object	yes	no	Diff
timestamp , source , ip	mixed	yes	no	Append-only — no update/delete path exists in the API

Database indexing (complete list)

Collection.field(s)	Index type	Purpose
persons.primary_phone , persons.primary_email	sparse B-tree	Dedup lookup at creation
persons.external_ids.id	B-tree	Integration lookups
organizations.institution_details.location	2dsphere	Map viewport/radius queries
organizations.institution_details.{district, institution_type, management_type}	compound B-tree	Map filter panel (district first — most selective)
relationships.{from_type,from_id} , {to_type,to_id}	compound B-tree	Timeline/360 lookups from either side
relationships.relationship_type , status	B-tree	MoU/relationship-status map filter
activities.related_person_id / related_organization_id / related_institution_id / related_opportunity_id	B-tree, one per field	Timeline queries
activities.timestamp	B-tree, descending	Recency sort, “activity recency” map filter
leads.stage , owner_id	compound B-tree	Pipeline board queries
opportunities.stage , owner_id , expected_close_date	compound B-tree	Pipeline board and forecasting
mous.end_date , status	compound B-tree	Expiry job
Atlas Search index: persons, organizations, leads, opportunities	Lucene text index	Global search (§13)

Definitive ERD





5. API CONTRACT

All routes prefixed `/api/v1`, require a bearer session token unless noted, return `{data, meta}` on success or `{error:{code,message}}` on failure. Errors used uniformly across every route below: **401** unauthenticated, **403** permission denied, **404** missing resource, **409** duplicate/conflict (dedup), **422** validation failure. List endpoints share `page`, `limit`, `sort`, `order`; resource-specific filters are listed per row.

Universal opportunity pipeline (macro-stage, referenced throughout)

Stage	Meaning
Discovered	Identified as a potential account/contact
Engaged	First contact made
Qualified	Confirmed fit and authority
Opportunity	Formally opened as a pipeline deal
Proposed	Proposal/quote sent
Negotiating	Terms under discussion
Won / Lost	Closed
Delivering	Won and in delivery (Project active)
Outcome	Delivery complete, outcome recorded
Renew / Expand / Refer	Post-outcome pipeline re-entry

A vertical's own stage list (e.g. Education admissions: Inquiry → Counselling → Application → Admission Offered → Enrolled) maps to one of these via `maps_to_macro_stage`, so cross-vertical pipeline reporting stays possible without forcing identical stage names everywhere.

Authentication

Method & Route	Purpose	Auth	Request	Response
POST /auth/login	Login	None	{email, password}	{user, token, expires_at}
POST /auth/logout	Invalidate session	Session	—	204
POST /auth/refresh	Rotate token	Refresh token	{refresh_token}	{token, expires_at}
POST /auth/password-reset/request	Send reset link	None	{email}	202
POST /auth/password-reset/confirm	Set new password	Reset token	{token, new_password}	204

Users

Method & Route	Purpose	Permission	Params/Body	Response
GET /users	List users	users:view (Admin)	?role, ?status	User[]
POST /users	Provision user	users:create (Admin)	{name, email, role_id}	User (invite sent)
PATCH /users/:id	Edit/deactivate	users:edit	{status, role_id,...}	User

People

Method & Route	Purpose	Permission	Params/Body	Response	Validation
GET /people	List/search	people:view	?q, ?role_context, ?district, ?tag, paginated/sortable	Person[]	—
POST /people	Create	people:create	Person fields (\$4)	Person	Server runs dedup check (phone/email exact match) first; 409 with candidate matches if found
GET /people/:id	Full profile	people:view	—	Person + relationships[] + recent activities[]	—
PATCH /people/:id	Edit	people:edit (own-owned only for Telecaller/Counsellor)	partial fields	Person	—
POST /people/:id/merge	Merge duplicate	people:edit (Admin only per wireframe)	{merge_into_id}	Person (canonical)	Losing record kept with dedupe_status:merged
GET /people/:id/timeline	Activity timeline	people:view	?type, ?from, ?to, ?page	Activity[] , paginated	—

Organizations & Institutions

Method & Route	Purpose	Permission	Params/Body	Response
GET /organizations	List/search	organizations:view	?category, ?q, ?parent_org_id	Organization[]
POST /organizations	Create	organizations:create	Organization fields	Organization
GET /organizations/:id	Profile	organizations:view	—	Organization + relationships + activities summary
PATCH /organizations/:id	Edit	organizations:edit	partial fields	Organization
GET /institutions	Institution list (table view)	institutions:view	?district, ?institution_type, ?management_type, ?relationship_status, ?mou_status	Institution[] (category=institution)
GET /institutions/map	Map data (\$10)	institutions:view	?bbox OR ?lat,lng,radius ; ?district, ?institution_type, ?management_type, ?relationship_status, ?mou_status, ?activity_recency, ?priority	Thin marker array: {institution_id, name, type, lat, lng, district, relationship_status, mou_status, last_activity_at, priority}[]
GET /institutions/:id	Lightweight profile (drawer)	institutions:view	—	Name, type, address, relationship_status, mou_status, last_activity_at
GET /institutions/:id/360	Full Institution 360	institutions:view	—	Full contract, \$10

Relationships

--	--	--	--	--

Method & Route	Purpose	Permission	Params/Body	Response
GET /relationships	List for an entity	relationships:view	?entity_type, ?entity_id (both required — always scoped, never a global list)	Relationship[]
POST /relationships	Create edge	relationships:create	{from_type, from_id, to_type, to_id, relationship_type, role, start_date}	Relationship
PATCH /relationships/:id	End/update	relationships:edit	{status, end_date, role,...}	Relationship

Leads & Opportunities

Method & Route	Purpose	Permission	Params/Body	Response
GET /leads	Pipeline list	leads:view	?stage, ?owner_id, ?source, ?vertical	Lead[]
POST /leads	Create	leads:create	Lead fields; creates Person first if new	Lead
POST /leads/:id/convert	Convert to Opportunity	leads:edit	{account_id?, contact_id?} (omit to reuse Lead's own person/org)	Opportunity + updated Lead
GET /opportunities	Pipeline board data	opportunities:view	?stage, ?owner_id, ?vertical	Opportunity[]
POST /opportunities	Create directly	opportunities:create	Opportunity fields	Opportunity
PATCH /opportunities/:id	Edit / change stage	opportunities:edit (value/probability: owner+Admin; Won/Lost: owner or manager)	partial fields	Opportunity — stage change emits OpportunityStageChanged

Activities, Tasks

Method & Route	Purpose	Permission	Params/Body	Response
POST /activities	Log activity against 1+ related entities	activities:create	{type, related_*_id(s), timestamp, duration, outcome, notes, next_action}	Activity — appears immediately on every related entity's timeline
GET /activities	Cross-entity feed ("my activities today")	activities:view	?actor_id, ?type, ?from, ?to	Activity[]
GET /tasks	My/assigned tasks	tasks:view	?assignee_id, ?status, ?due_before	Task[]
POST /tasks	Create follow-up	tasks:create	{related_activity_id?, related_entity_type/id, assignee_id, due_date, priority}	Task
PATCH /tasks/:id	Complete/reassign	tasks:edit	partial fields	Task

MoUs

Method & Route	Purpose	Permission	Params/Body	Response
GET /mous	List	mous:view	?organization_id, ?institution_id, ?status, ?expiring_within_days	MoU[]
POST /mous	Create	mous:create	{organization_id/institution_id, scope, vertical, owner_id, start_date, end_date}	MoU (status=proposed)
PATCH /mous/:id	Advance lifecycle / renew	mous:edit ; approve/sign-off = Admin/Founder only	{status, end_date,...}	MoU
POST /mous/:id/renew	Start renewal (clones into new draft)	mous:edit	—	New MoU (renewed_from_id set)

Education

Method & Route	Purpose	Permission	Params/Body	Response
GET /courses, POST /courses	Catalog	education:view / :create	—	Course[] / Course
GET /students, GET /students/:id	List/profile (wraps Person + academic details)	education:view	?course_id, ?status	Student[] / Student+Person
POST /enrollments	Enroll a Student	education:create	{student_id, course_id, batch}	Enrollment
POST /attendance	Log session attendance	education:create (Trainer: own batch)	{enrollment_id, session_date, status}	Attendance
POST /daily-progress	Log progress entry	education:create	{enrollment_id, date, notes, score}	DailyProgress
GET /recommendations	Suggested next actions	education:view	?student_id	Recommendation[]

Projects & Reports

Method & Route	Purpose	Permission	Params/Body	Response
GET /projects, PATCH /projects/:id	Delivery status tracking	projects:view / :edit	?opportunity_id	Project[] / Project
GET /reports/dashboard	Executive Dashboard KPI set	reports:view (financial KPIs gated by F permission)	?from, ?to, ?vertical, ?branch, ?owner, ?region	KPI summary object: revenue_by_vertical, pipeline_summary, collections, mou_status_counts, task_queue, activity_feed

Search, Files, Notifications

Method & Route	Purpose	Permission	Params/Body	Response
GET /search	Global search	any authenticated user (result set filtered per-resource permission)	?q	Grouped results by entity type (Person/Org/Institution/Opportunity/MoU), badge per kind, recent + fuzzy + exact-ID match
POST /documents	Upload	<resource>:edit on the parent entity	multipart file + {attached_to_type, attached_to_id}	Document (signed, time-limited URL returned)
GET /documents/:id/download	Download	<resource>:view on the parent entity	—	Signed redirect URL
GET /notifications	List for current user	self	?read	Notification[]
PATCH /notifications/:id	Mark read	self	{read_at}	Notification

Every “major wireframe action” maps to a route above: creating any record, converting a lead, changing an opportunity stage, logging an activity from any 360 screen, creating/renewing/advancing an MoU, filtering and selecting on the map, enrolling/marking attendance/logging progress, and drilling into a dashboard KPI. No screen action in the wireframe is left as “backend handles this” without a named endpoint.

6. SCREEN / ROUTE SPECIFICATION

Route inventory

Route	Purpose	Data required	API calls	Permissions	Primary actions	Secondary	Loading	Empty state	Error
-------	---------	---------------	-----------	-------------	-----------------	-----------	---------	-------------	-------

						actions	state		state
/dashboard	Founder/Exec KPI overview — default landing page	Filtered KPI set	GET /reports/dashboard	reports:view ; financial KPIs hidden for Telecaller/Trainer/Counsellor	Change filters (quarter/vertical/branch/owner/region), drill into KPI	Complete/reassign task, open activity row	Skeleton KPI cards	“No data for this filter combination.”	Retry banner; last-known values shown stale-flagged
/people	People list	Paginated Person[]	GET /people	people:view	Search, filter (role/org/stage/owner/location/last activity), open record	Quick Create Person	Skeleton rows	“No search results” → suggest quick-create	Retry banner
/people/:id	Person 360	Person + relationships + timeline + opportunities + education history + employment + documents + notes	GET /people/:id, /people/:id/timeline	people:view ; edit = Sales/Admin, own-owned for Telecaller/Counsellor; delete = Admin only	Log Activity, Edit, Add role/link organization, create opportunity	Merge duplicate (Admin)	Skeleton tabs	“Not linked to any organization yet.”	Retry banner
/organizations	Organizations list	Paginated Organization[]	GET /organizations	organizations:view	Search, filter (type/industry/relationship/owner), open record	Quick Create Organization	Skeleton rows	Empty search → suggest quick-create	Retry banner
/organizations/:id	Organization 360	Org + hierarchy + buying committee + relationship health + contacts + opportunities + contracts/payments + projects + MoUs + timeline	GET /organizations/:id	organizations:view ; Finance tab hidden from Telecaller/Trainer/Counsellor	New Opportunity, Edit, Add contact	Link parent/subsidiary, create MoU/project, log activity	Skeleton tabs	No subsidiaries → hierarchy card hidden; no buying committee → “Add contacts to this deal”	Retry banner
/institutions	Institution list (table, alt. to map)	Paginated Institution[]	GET /institutions	institutions:view	Filter (district/type/relationship/MoU/priority), open Institution 360	Switch to map view	Skeleton rows	“0 institutions match — widen your filters”	Retry banner
/institutions/map	Institution Map	Thin marker set for viewport+filters	GET /institutions/map	institutions:view	Filter, pan/zoom, select marker → drawer, “Open Institution 360”	Cluster expand, search institution by name	Skeleton map / loading pins	“0 institutions match your filters”	Retry banner; map tiles fail → static list fallback
/institutions/:id	Institution 360	Full contract (\$10)	GET /institutions/:id/360, /:id/timeline	institutions:view ; Counsellor/BD: view+create for owned/unowned in branch; Admin: reassign owner; Finance: view only	Call/WhatsApp/Email, Create MoU, Create Activity	Reassign owner (Admin)	Skeleton tabs, header loads before slower tabs	“No activity yet — log the first touchpoint.”	Retry banner
/leads	Lead pipeline list	Paginated Lead[]	GET /leads	leads:view	Filter (source/vertical/stage/owner/score), convert	Quick Create Lead	Skeleton rows	“No leads match”	Retry banner
/leads/:id	Lead detail	Lead + Person context	GET /leads/:id	leads:view / :edit	Convert to Opportunity	Log activity	Skeleton	—	Retry banner
/opportunities	Pipeline board (Kanban, list toggle)	Opportunities grouped by stage	GET /opportunities	opportunities:view	Drag between stages, open detail	Toggle table view	Skeleton columns	Empty stage column: “No opportunities in this stage”	Retry banner
/opportunities/:id	Opportunity detail	Deal + related + delivery chain	GET /opportunities/:id	opportunities:view ; value/probability edit = owner+Admin; Won/Lost = owner/manager; Telecaller = create lead only, no opp edit	Log Activity, Mark Won (requires reason if Lost)	Generate proposal, spin up Project on Won	Skeleton	—	Retry banner
/mous	MoU list	Paginated MoU[]	GET /mous	mous:view	Filter (All/Active/Expiring soon/Expired/No MoU), open detail	Create MoU	Skeleton rows	“No MoUs match”	Retry banner
/mous/:id	MoU detail	MoU + document + activities covered	GET /mous/:id	mous:view ; create/edit = Sales/Admin/Partnerships; approve/sign-off = Admin/Founder only	Start Renewal, Upload signed doc	Advance lifecycle stage	Skeleton	—	Retry banner
/activities , /tasks	Cross-record activity/task inbox	Activity[] / Task[]	GET /activities, GET /tasks	activities:view , tasks:view	Complete task, log activity	Reassign task	Skeleton	“No activities logged today”	Retry banner
/education/students , /education/students/:id	Student list / Student 360	Student(+Person) list/detail, progress, attendance	GET /students[, /:id]	education:view ; Trainer = own batch only; Student self-service = own dashboard	Enroll, log attendance/progress, run recommendation, check in	Issue certificate, create placement record	Skeleton tabs	Recommendation empty: “Complete interests & goals to see matches.”; at-risk view empty: “No students below threshold”	Retry banner
/education/admissions	Admissions pipeline (funnel)	Stage counts	GET /leads? vertical=education	education:view	Advance stage	—	Skeleton funnel	“No enquiries this period”	Retry banner

Screen → Component → API → Entity → Database mapping

Screen	Key components	API	Entity	Collection(s)
Dashboard	AppShell, KPI Card, ActivityTimeline (recent activity table)	GET /reports/dashboard	(aggregated)	opportunities , mous , payments , activities , daily_metrics
People list / Person 360	DataTable, FilterPanel, RecordHeader, RecordTabs, RelationshipPanel, ActivityTimeline, ActivityComposer	GET/POST /people, GET /people/:id/timeline, POST /relationships, POST /activities	Person, Relationship, Activity	persons , relationships , activities
Organizations list / Org 360	DataTable, FilterPanel, RecordHeader, RecordTabs, ContactCard	GET/POST /organizations, GET /organizations/:id	Organization, Relationship, MoU, Opportunity	organizations , relationships , mous , opportunities
Institution list	DataTable, FilterPanel	GET /institutions	Institution (Organization subtype)	organizations
Institution Map	Map, MapMarker, MapCluster, FilterDrawer, InstitutionDrawer	GET /institutions/map, GET /institutions/:id	Institution, Relationship, MoU	organizations , relationships , mous
Institution 360	RecordHeader, RecordTabs, RelationshipPanel, ActivityTimeline,	GET /institutions/:id/360, POST /activities, POST /mous	Institution, Relationship, Activity, MoU, Opportunity	organizations , relationships , activities , mous , opportunities

People	VCEDAX	VCEDAX	VCEA	VCEA (own)	VCEA (own)	V	V	VCEA	V	V
Organizations	VCEDAX	VCEDAX	VCEA	V	V	V	V	VCEA	V	VC
Institutions	VCEDAX	VCEDAX	VCEA	V	VCEA (own/unowned in branch)	V	V	VCEA	V	V
MoUs	VCEDAX	VCEDAX	VCEA	—	VCEA	—	V	VCEA	V	V
Leads	VCEDAX	VCEDAX	VCEA	VCEA (own)	VCEA	—	—	VCEA	—	VC
Opportunities	VCEDAF	VCEDAX	VCEA (value/prob: owner+Admin only)	V (create lead only, no opp edit)	VCEA	—	V	VCEA	VF	V
Education (Students/Enrollment)	VCEDAX	VCEDAX	V	V	VCEA	VCE (own batch)	—	VCEA	V	V
Projects	VCEDAX	VCEDAX	V	—	—	VCE	VCEA	V	VF	V
Payments	VCEDAF	VCEDAF	—	—	—	—	V	V	VCEDAF	—
Reports/Dashboard	VXF	VX	V	V	V	V	V	V	VXF	V
Documents	inherits parent-entity view / edit permission									

Implementation decision: the exact per-cell grants above are the Phase 1 default inferred from the wireframe's own permission callouts and role names (both sources agree on the shape but the wireframe's own condensed matrix does not cover every resource). Confirm with the founder before locking **Financial visibility (F)** and **Delete (D)** in particular — these are the two columns with the highest business risk if wrong, and the tech spec explicitly flags them as needing sign-off.

Sensitive-data restrictions

- Financial KPIs (revenue, collections, payment amounts) are hidden from Telecaller, Trainer, and Education Counsellor roles on the Dashboard and on Organization 360's Contracts & Payments tab.
- Student PII (attendance/progress detail) is visible only to Education Counsellor, the assigned Trainer (own batch only), Admin, and the student's own self-service view — never to Sales/Telecaller/Marketing.
- Document download requires the `view` permission on the document's parent entity, checked server-side per request (\$13), not inferred from the document's own metadata.

9. ACTIVITY / EVENT / AUTOMATION MODEL

Universal activity model

One `activities` collection (\$4), typed by `type` (call, whatsapp, email, meeting, visit, workshop, training, event, follow_up, proposal, payment, mou_activity, research, survey, other), carrying up to five optional `related_*_id` foreign keys (person, organization, institution, lead, opportunity/project).

How one activity appears on multiple timelines without duplication: a timeline view (Person, Organization, Institution, Opportunity, Student) never stores its own activity list — it queries the single `activities` collection filtered on its own `related_*_id` field. Logging one visit that involved a Person (e.g. a Placement Officer) at an Institution, tied to an Opportunity, is a single Activity document with all three `related_*` fields set — it satisfies three timeline queries from one write, with no fan-out or copy.

- Creation:** `ActivityComposer`, opened from any 360 page, pre-fills the relevant `related_*_id` ; `POST /activities` .
- Editing:** `outcome/notes/next_action` editable by the actor or owner; `timestamp/type` immutable after creation (audit integrity).
- Ownership:** `owner_id` defaults to `actor_id` , reassignable by Admin.
- Next action:** setting `next_action` on an Activity auto-creates a linked `Task` (`related_activity_id` set), surfaced in the Activities/Tasks hub and on the originating 360 page.

Domain events

Events are a side-effect broadcast for asynchronous consumers (notifications, audit, cache invalidation, analytics) — the MongoDB write is always the source of truth; this is not a full event-sourcing architecture.

Event	Trigger	Condition	Action	Recipient	Failure behavior
PersonCreated	POST /people	—	Write AuditLog entry	— (audit only)	Retry queue; if it never succeeds, flagged in an admin audit-gap report
RelationshipCreated	POST /relationships	—	Invalidate Institution/Org/Person 360 cache; write AuditLog	—	Same as above
LeadCreated	POST /leads	—	Notify assignee	Lead owner	Retry queue; failed send surfaces as a Task
LeadConverted	POST /leads/:id/convert	—	Write AuditLog; feed daily pipeline rollup	Analytics job	—
LeadUntouched	Scheduled check	No activity in N days (configurable)	Notify assignee; escalate to manager after further delay	Lead owner, then manager	—
OpportunityStageChanged	PATCH /opportunities/:id (stage field changes)	—	Notify owner; notify Finance if stage=Won	Opportunity owner (+ Finance on Won)	—
ActivityCreated	POST /activities	—	Invalidate related timelines' cache; auto-create Task if <code>next_action</code> set	Related entity owners	—
MoUExpiring	Scheduled daily job	<code>end_date</code> within 90/60/30/7 days	Set <code>status=expiring</code> (idempotent); notify owner, escalating with proximity	MoU owner	—
MoUExpired	Scheduled daily job	<code>end_date</code> passed, not renewed	Set <code>status=expired</code>	MoU owner	—
StudentEnrolled / StudentCompleted	POST /enrollments, status PATCH	—	Feed analytics rollup, trigger Recommendation job	—	—
PaymentReceived	Payment webhook	—	Notify Finance + Opportunity owner	Finance, Opportunity owner	Reconciliation job flags mismatches
TaskOverdue	Scheduled check	<code>due_date</code> passed, <code>status=complete</code>	Notify assignee; escalate to manager after 2 days overdue	Assignee, then manager	—
StudentAtRisk	Scheduled check	Attendance/progress below configured threshold	Notify Education Counsellor	Counsellor	—
InstitutionInactive	Scheduled check	No activity in N days	Notify Institution owner (low priority)	Institution owner	—
ProposalPending	Scheduled check	No response N days after proposal-type Activity	Notify Opportunity owner	Opportunity owner	—

Not every write emits an event — routine reads and minor edits (e.g. correcting a typo in a Note) do not; events are reserved for state transitions another part of the system genuinely reacts to.

Notifications

Stored in a lightweight `notifications` collection (`recipient_id`, `type`, `payload`, `read_at`) so `NotificationCenter` shows history, not just a toast. Email channel reuses the same event payload — not a

separately maintained template flow. Full trigger/recipient/channel/priority/action table is in §13.

10. INSTITUTION MAP + INSTITUTION 360

This is the flagship Phase 1 flow — fully specified end to end.

Scope

Tamil Nadu → District → Institution. Map defaults to **Madurai district** on load. Institution types: School, College, University, Other. Management types: Government, Private, Aided, Autonomous (Other).

Institution schema (repeated from §4 for this section’s completeness)

`institution_details`: `institution_type` (school|college|university|training_institution|other), `management_type` (government|private|aided|autonomous), `location` (GeoJSON Point, **2dsphere** indexed), `address`, `district`, `taluk`, `external_identifier` (authoritative directory code, e.g. AISHE/UDISE — never fabricated, see Implementation Decision below). `relationship_status` and `mou_status` are **not stored on the institution document** — they are computed at query time from the latest Relationship/MoU record, so they can never drift out of sync with the underlying record.

Implementation decision: the wireframe implies institution data exists at meaningful volume across Madurai district (and eventually Tamil Nadu). No unverified institution rows should be seeded into the schema as placeholder data. Populate district/institution directories from an authoritative source (e.g. state education department listings, AISHE/UDISE) as a data-loading task — which source to license is a product decision, not a schema decision; the schema accepts any source once identified.

Map query architecture

Concern	Approach
What’s queried	<code>GET /institutions/map</code> with a viewport bbox OR center+radius, plus the active filter set (district, type, <code>management_type</code> , <code>relationship_status</code> , <code>mou_status</code> , <code>activity_recency</code> , priority) — a single <code>\$geoWithin / \$nearSphere</code> query combined with the other filters in one pass, never fetched-then-filtered client-side
What’s indexed	2dsphere on <code>institution_details.location</code> ; compound B-tree on <code>district</code> , <code>institution_type</code> , <code>management_type</code> (district first — most selective, matches the wireframe’s district-first filter UX)
What’s cached	Per-district institution counts and the default Madurai marker set (Redis, 5-15 min TTL)
What’s computed dynamically	<code>relationship_status</code> , <code>mou_status</code> per marker — from the latest Relationship/MoU record, at query time
What’s precomputed	Marker clustering for zoomed-out views: a scheduled job buckets institutions into geohash cells and stores cell-level counts; a zoomed-out viewport returns cluster bubbles from this precomputed table instead of clustering thousands of points per request. Clicking into a cluster switches to the live per-marker query at that tighter viewport
Viewport query	<code>GET /institutions/map?bbox=...</code> for pan/zoom; <code>?lat,lng,radius</code> for the drawer’s “15 km radius around selected point” mode
Marker payload	Deliberately thin: { <code>institution_id</code> , <code>name</code> , <code>type</code> , <code>lat</code> , <code>lng</code> , <code>district</code> , <code>relationship_status</code> , <code>mou_status</code> , <code>last_activity_at</code> , <code>priority</code> } — no contacts, activities, or documents, so payload size stays constant regardless of how much detail exists on any one institution
Clustering	Server-side geohash bucketing at zoomed-out levels; client-side marker virtualization for the visible viewport at zoomed-in levels
Marker rendering	Shape = institution type (● school, ■ college, ◆ university); ring color = relationship status; cluster badge shows count at zoom-out

Selection behavior

- User applies filters (district/type/management/relationship/recency/priority) → URL state updates → `GET /institutions/map` refetches.
- Click a marker → `GET /institutions/:id` (lightweight profile) → `InstitutionDrawer` opens with name, type, address, `relationship_status`, `mou_status`, `last_activity_at`, distance from selection point, quick-action chips (Call/WhatsApp/Email), `Create Activity` and `Open Institution 360` → buttons.
- Click a cluster → expand to individual markers at that tighter viewport (re-query, no separate endpoint).

Institution Drawer

Renders entirely from the `GET /institutions/:id` response already fetched for step 2 above — no extra call.

Institution 360 — backend data contract (GET /institutions/:id/360)

Section	Directly stored	Referenced (fetched on demand)	Aggregated/computed
Profile	<code>organizations</code> document (<code>category=institution</code>)	—	—
Contacts	—	<code>persons</code> via <code>relationships</code> where <code>to_id=institution</code>	Role label from <code>relationship.role</code>
Relationships	—	<code>relationships</code> collection	Relationship strength computed from recent activity frequency
Activities / Timeline	—	<code>activities</code> where <code>related_institution_id=this</code>	Sorted, paginated by timestamp
MoUs	—	<code>mous</code> where <code>institution_id=this</code>	<code>mou_status</code> shown = latest MoU’s status
Opportunities / Projects	—	<code>opportunities / projects</code> where <code>account_id=this</code>	Pipeline value sum (computed, not stored)
Programs / Students / Alumni	—	<code>enrollments</code> joined through <code>students</code> , where linkage exists via <code>studied_at</code> Relationship or program delivery	Enrolled/alumni counts computed at request time
Documents	—	<code>documents</code> where <code>attached_to=institution</code>	—
AI insights	—	N/A in Phase 1	Placeholder section only — rule-based reasons (e.g. “4,800 students across 3 relevant departments,” “no active MoU,” “3 previous activities logged”) may render as a static, clearly-labeled insight card; no live model runs (\$16 boundary)

The 360 header/profile renders first; Timeline, MoUs, Opportunities, and Programs/Students tabs load independently and lazily (each is its own query) so the record header never waits on the slowest section.

Institution Activity Timeline

Filter chips: All / Calls / Meetings / Campus Events / MoU / Proposals. Same polymorphic mechanism as §9 — queries `activities` where `related_institution_id = this`, chronological, paginated.

End-to-end journey (fully specified, step by step)

Step	Frontend action	Backend/API	Data returned	Event emitted	Permission check
1. Filter	Set district=Madurai, relationship=None relationship	<code>GET /institutions/map?district=Madurai&relationship_status=none</code>	Thin marker list	—	<code>institutions:view</code>
2. Click institution	Click a College marker	<code>GET /institutions/:id</code>	Lightweight profile	—	<code>institutions:view</code>
3. Drawer opens	Renders from step 2’s data	—	—	—	—

4. Open Institution 360	Click “Open Institution 360 →”	GET /institutions/:id/360	Full contract above	—	institutions:view
5. Notice inactivity	UI surfaces “no activity in 74 days” from the profile/relationship-summary card	(already loaded)	—	—	—
6. AI-flagged suggestion (rule-based)	AIInsightCard shows “Schedule a meeting with the Placement Officer”	(computed at 360 load, not a separate model call)	—	—	—
7. Create Activity	Click “Create Activity from this” → ActivityComposer prefilled with related_institution_id	POST /activities	New Activity	ActivityCreated	activities:create
8. Timeline updates	ActivityTimeline re-fetches / optimistically prepends	(client cache invalidation)	—	—	—
9. Create follow-up	Composer’s next_action field	POST /tasks {related_activity_id} (implicit, or explicit “Create follow-up”)	New Task	—	tasks:create
10. MoU / Opportunity	Click “Create MoU” or “New Opportunity” from Institution 360 header	POST /mous or POST /opportunities (institution_id / account_id prefilled)	New MoU/Opportunity	MoUCreated / (opportunity created)	mous:create / opportunities:create

This is the exact MAP → FILTER → CLICK INSTITUTION → INSTITUTION SUMMARY → INSTITUTION 360 → TIMELINE → CREATE ACTIVITY → CREATE FOLLOW-UP → MoU/OPPORTUNITY journey required, with every step’s frontend action and backend call named.

11. MoU

Fields

_id , organization_id / institution_id (at least one required), scope , vertical , owner_id , document_id (→ Document, the signed file), start_date / end_date , status , renewed_from_id .

Lifecycle

proposed → negotiating → approved → signed → active → expiring → expired | renewed

- Transitions proposed → negotiating → approved → signed → active are user-driven (PATCH /mous/:id); approve/sign-off restricted to Admin/Founder (§8).
- A **scheduled daily job** checks active MoUs’ end_date : at 90/60/30/7 days out it idempotently sets status=expiring and emits MoUExpiring .
- Past end_date with no renewal, the same job sets status=expired .
- **Renewal creates a new MoU** (status=proposed , renewed_from_id set) rather than editing the expired one — the expired MoU’s own status becomes renewed once its successor is signed. History is preserved as a chain, matching multiple-MoUs-per-institution support.

Relationships

MoU → Organization/Institution (required), → Document (signed file), → Activity (activities logged “under” the MoU, e.g. placement drives, workshops — related via the Institution/Organization the MoU targets), → Opportunity (a commercial deal can reference the MoU that enabled it).

Documents

Signed MoU file stored via the documents collection/object storage pattern (§13); mous.document_id references it. Re-signing (e.g. on renewal) creates a new Document record.

Expiry

Fully automated (see Lifecycle above) — no manual daily check required by staff; the notification system (§9/§13) surfaces expiring MoUs proactively at 90/60/30/7-day thresholds with escalating priority.

Permissions

Create/edit: Sales, Admin, Partnerships-tagged Sales users, Education Counsellor. Approve/sign-off: Admin, Founder only. View: all roles except none excluded per §8 matrix (Telecaller has no MoU access; Trainer has no MoU access).

Notifications

MoUExpiring (90/60/30/7 days, escalating) and MoUCreated / MoU signed → institution.mou_status recomputed — see §9 event table and §13 notification table.

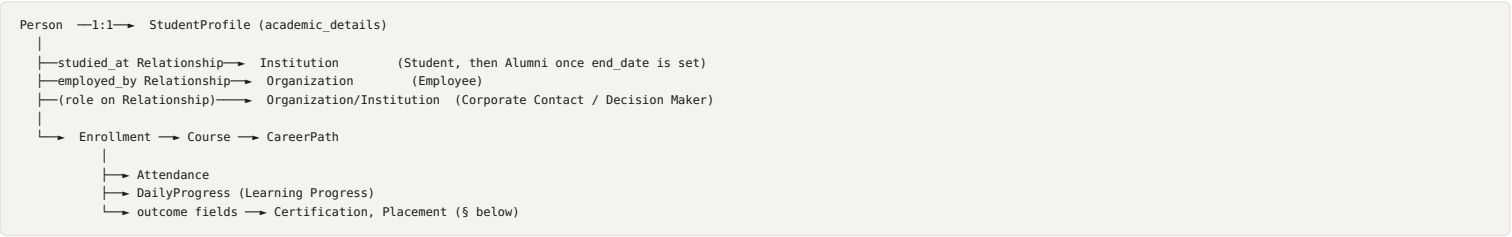
Appearance in Institution 360 and timelines

- Institution 360’s **MoUs tab** lists all MoUs for that institution (mous where institution_id=this); the institution’s displayed mou_status badge always reflects the **latest** MoU’s status (computed, not stored).
- MoU-type activities (activity.type = mou_activity) appear on both the MoU’s “Activities covered under this MoU” section and the Institution’s own Activity Timeline, via the same polymorphic related_institution_id mechanism (§9) — no duplication.
- The MoU list screen supports the wireframe’s filter chips: All / Active / Expiring soon / Expired / No MoU (institutions) — the last one is a join against Institutions with no mous record, not a filter on the mous collection itself.

12. EDUCATION INTEGRATION

Boundary restated: Phase 1 Education is CRM-integrated capability only — Students, Courses, Career Paths, Recommendations, Enrollments, Attendance, Daily/Learning Progress, Certification, Placement, Alumni. No video hosting, lesson authoring, quiz/assignment engines, question banks, SCORM, or LMS content analytics ship in Phase 1 (§1 OUT OF SCOPE).

No second person database



- **Student** = a Person + a 1:1 StudentProfile sub-record (academic_details : interests, qualification, career goal — feeds the recommendation engine) + one or more Enrollments. It is never a standalone identity.
- **Alumni** = the same Person’s studied_at Relationship gets an end_date ; no new record is created, and no data is lost — the Person carries the full history as an ordered set of time-bounded Relationships.
- **Certification** = an outcome field/Activity type on the Enrollment/Person (e.g. activity.type capturing certificate issuance, or a certified_in Relationship to the issuing Institution) — not a new parallel entity; it reuses the same Activity/Relationship machinery as everything else in the model.
- **Placement** = an Opportunity/Activity + a new employed_by Relationship from the same Person to the hiring Organization, created at the moment the student is placed — this is exactly the “Student → Alumni → Employee” chain the whole architecture exists to support, on one Person record.
- **Employer/Corporate Contact** = the same Person, now also carrying a decision_maker_for /role-bearing Relationship to an Organization, fully independent of (but co-existing with) their Alumni Relationship — enabling the “cross-vertical bridge” case shown in the wireframe (a Kaizen alumnus who becomes the corporate contact for a training deal at their employer).

What’s preserved from the existing Education CRM, and how it connects

Existing capability	Preserved as	Connects to universal CRM via
Student records	Person + StudentProfile	person_id (1:1)
Admissions pipeline	Lead with vertical=education , stages mapped to universal macro-stages via maps_to_macro_stage	Lead → Person, Lead → Opportunity on conversion
Course/Career Path catalog	courses / career_paths collections, unchanged in behavior	Enrollment → Course → CareerPath
Enrollment	enrollments collection, student_id → students.person_id	Student (Person-backed)
Attendance, Daily Progress	Referenced (not embedded) per Enrollment (\$4 — unbounded growth reasoning)	Enrollment
Recommendation engine	Rule-based weighted factors (Interest Match 30%, Qualification 20%, Career Goal 20%, Duration 15%, Difficulty 10%, Industry 5%) — ≥85% Recommended, 40–84% Available, <40% Hidden. Not a model — this is explicitly rule-based, matching the AI-readiness boundary in \$1/\$16	students.academic_details → courses
Learning Dashboards	Three role-based views: Student (today’s session, attendance, current topic, completion %, rating, difficulties, tomorrow’s plan, streak), Trainer (batch roster, attendance grid, progress by student, at-risk flags, session ratings, topics completed), Admin/Learning (batch performance, attendance trend, completion rate, drop-off analysis, trainer performance, learning outcomes)	Same enrollments / attendance / daily_progress data, role-scoped query

Implementation decision: whether every historical education-only record (pre-migration) gets backfilled into Person/Relationship, or only new records go through the universal model going forward, is a migration-scope decision. Recommend backfilling at minimum current/active students and staff at Phase 1E build time; full historical backfill can run as a background migration job after Phase 1E ships (\$21).

13. SEARCH / FILES / NOTIFICATIONS

Search

- **Entities indexed:** Person (full_name , phone , email), Organization/Institution (name), Lead/Opportunity (identifying fields), MoU (scope), Course (name), Student (via linked Person).
- **Engine:** MongoDB Atlas Search (Lucene-based, same cluster) — no separate Elasticsearch/OpenSearch cluster for Phase 1 query volume.
- **Behavior:** exact match (phone/email/ external_id via term-level index) for dedup and direct lookup; fuzzy name matching (Lucene fuzzy) for GlobalSearch ; prefix/autocomplete as the user types.
- **Filters:** GlobalSearch results are grouped by entity type with a badge (Institution/Person/Opportunity/MoU...), showing recent + fuzzy + exact-ID matches together.
- **Route:** GET /search?q= (\$5).
- Revisit a dedicated search cluster only if relevance-tuning complexity across many entity types, or query volume, outgrows Atlas Search’s limits.

Files

- **Storage:** object storage (S3-compatible); binaries are never stored in MongoDB documents.
- **Metadata:** documents collection (filename, storage_key, mime_type, size, attached_to_type/id, version, document_group_id, uploaded_by), \$4.
- **Attachment relationships:** polymorphic attached_to_type/id , so a document can attach to a Person, Organization/Institution, Opportunity, or MoU without a per-entity documents table.
- **Permissions:** access enforced at the API layer — a signed, time-limited download URL is issued only after a permission check on the parent entity (e.g. an MoU document requires mous:view); the storage bucket itself is not entity-aware.
- **Versioning:** each re-upload creates a new documents record with an incremented version , sharing a document_group_id , rather than overwriting the storage object — prior versions remain retrievable for audit.

Notifications

Trigger	Recipient	Channel	Priority	Action
Lead assigned	Assignee	In-app + email	Normal	Open Lead
Lead untouched (no activity in N days)	Assignee, then manager after further delay	In-app	High	Open Lead, log activity
Follow-up due (Task due_date reached)	Assignee	In-app + email	Normal	Complete Task
Task overdue	Assignee, then manager after 2 days overdue	In-app	High	Complete/reassign Task
Opportunity stage changed	Owner (+ Finance if stage=Won)	In-app	Normal	Open Opportunity
MoU expiring (90/60/30/7 days)	MoU owner	In-app + email	Escalating with proximity	Renew or close out MoU
Payment pending	Finance, Opportunity owner	In-app	Normal	Follow up
Student at risk (attendance/progress below threshold)	Education Counsellor	In-app	High	Log intervention activity
Institution inactive (no activity in N days)	Institution owner	In-app	Low	Schedule outreach
Proposal pending (no response in N days)	Opportunity owner	In-app	Normal	Follow up

Notifications persist in a notifications collection (not just a toast), so NotificationCenter shows history; email reuses the same event payload rather than a separately maintained template flow. N thresholds (lead-untouched days, at-risk threshold, inactive-institution days, proposal-pending days) are configurable, not hardcoded — a Settings-level config in a later phase, environment-variable-defaulted in Phase 1 (see Implementation Decision, \$21).

14. INTEGRATIONS

System	Purpose	Direction	Data	Trigger	Auth	Source of truth	Failure behavior	Phase
WhatsApp Business API	Activity/communication logging	Bi-directional	Message content → Activity/Communication; outbound template sends	Webhook (inbound), API call (outbound)	API key/OAuth	Kaizen CRM	Retry queue; failed sends surface as a Task	Phase 1
Telephony (click-to-call / call logging)	Call activity logging	Bi-directional	Call metadata → Activity (duration, outcome)	Webhook on call end	API key	Kaizen CRM	Missed webhook = manual activity log fallback	Phase 1
Email (SMTP/provider API)	Communication logging	Bi-directional	Thread content → Communication; logged as Activity	Send action, inbound webhook	OAuth/API key	Kaizen CRM	Bounce handling → notify sender	Phase 1
Payment gateway / UPI	Payment confirmation	Inbound	Payment confirmation → Payment record	Webhook on payment success	Signed webhook secret	Payment gateway (financial fact), CRM (record)	Reconciliation job flags mismatches	Phase 1 (Finance-critical)
Website forms	Lead capture	Inbound	Form submission → Lead	Webhook	Signed form token	Kaizen CRM	Dedup check runs same as manual Lead creation	Phase 1
Geocoding / maps tiles	Institution location resolution, map rendering	Outbound	Address → lat/lng at institution creation; tile rendering	On institution create/edit; map render	API key	Kaizen CRM (institution_details.location)	Manual lat/lng entry fallback if geocoding fails	Phase 1

Calendar (Google/Outlook)	Meeting sync	Bi-directional	Meeting Activity ↔ calendar event	On Activity create/edit	OAuth	Calendar provider	Sync conflict = last-write-wins with a warning	Phase 2
LMS	Attendance/progress/completion sync	Bi-directional	Attendance, progress, completion → DailyProgress/Enrollment	Scheduled sync or webhook	API key	LMS (for content-delivery facts)	Sync log + manual reconciliation view	Phase 2
Marketing systems	Lead/contact export for campaigns	Outbound	Lead/Contact export	Scheduled or on-demand export	API key	Kaizen CRM	—	Phase 2
HRMS	Employee provisioning	Inbound	Employee → User/Person provisioning	Scheduled sync	API key	HRMS	Manual override always available	Future
ERP / Accounting	Invoicing/payments sync	Bi-directional	Invoices, payments	Scheduled sync	API key	ERP (for accounting facts)	Reconciliation report	Future

MVP (Phase 1) integrations are exactly the ones the wireframe’s activity logging and Institution Map cannot function acceptably without: WhatsApp/telephony/email logging, payments, website-form leads, and geocoding. Everything marked Phase 2/Future is real and specified but not built in Phase 1 — no Phase 1 workflow depends on them, and building them now would delay the core CRM.

15. SECURITY / AUDIT / DATA GOVERNANCE

Engineering checklist — not a restatement of legal policy.

Authentication - [] Email+password login, bcrypt/argon2 password hashing. - [] Session token (JWT or opaque) with server-side revocation list; 8-hour sliding expiry, forced re-login after 30 days idle. - [] Password reset via emailed, time-limited (60 min) token. - [] User provisioning: Admin creates User + Role → invite email with set-password link. - [] User deactivation is soft (`status=inactive`) — blocks login, hides from assignee pickers, but historical `owner_id` / `actor_id` references remain intact for audit continuity. - [] MFA: not built in Phase 1 unless the organization mandates it before build start (§21 open decision).

Authorization - [] Every API route enforces the permission matrix (§8) server-side — never trust client-side role checks alone. - [] Record-level narrowings (own-owned scoping for Telecaller/Counsellor) enforced in the same middleware layer as role checks, not bolted on per-route.

PII handling - [] Person PII (name, phone, email, address) and Student academic/attendance data are the highest-sensitivity fields in this system; access gated by §8’s matrix. - [] Export (`X` permission) is itself logged as an audit event, distinct from view.

Student data - [] Attendance/progress detail visible only to Education Counsellor, the assigned Trainer (own batch), Admin, and the student’s own self-service view.

Financial data - [] Financial visibility (`F`) is a distinct permission from `View` (§8) — Sales can work a deal without seeing margin/payment detail unless explicitly granted `F` . - [] Payment webhook payloads validated against a signed secret before being trusted.

Audit logging - [] Every write to a governance-relevant collection (Person, Organization, Institution, Relationship, Opportunity, MoU, Payment, Student, Enrollment) writes a corresponding `audit_logs` entry: `subject_type`, `subject_id`, `actor_id`, `action`, `before{}`, `after{}`, `timestamp`, `source`, `ip` . - [] `audit_logs` is append-only — no update or delete path exists in the API for it.

Data deletion - [] Soft-delete via `deleted_at` is the default for every entity. - [] Hard delete/anonymize is reserved for governance-mandated erasure (e.g. a departed student/employee’s PII deletion request), implemented as a background job keyed on `deleted_at` , gated by Admin. - [] Merge operations (§4 Person model) never hard-delete the losing record — it’s kept with `dedupe_status:merged` , `merged_into` set, so historical references and audit trails stay valid.

Data export - [] Any bulk export (`X` permission) is logged with actor, scope, and record count.

Backups - [] Daily automated Atlas backups with point-in-time recovery. - [] Retention window per Kaizen’s data-retention policy (§21 open decision — legal/compliance sign-off required, not a build blocker for backups themselves).

Secrets - [] API keys and integration credentials (WhatsApp, telephony, email, payment gateway, geocoding) live in a managed secrets store — never in source control or plain config.

Encryption - [] TLS in transit everywhere. - [] Atlas-managed encryption at rest.

Retention - [] Retention period is a compliance/legal decision requiring sign-off before the hard-delete/anonymize job is built (§21) — the schema already supports it structurally via `deleted_at` .

Implementation decision: DPDP (or equivalent) compliance requirements beyond what’s captured above (e.g. consent capture, data-subject-request workflow specifics) are a legal-review deliverable, not something either source document fully specifies — flagged, not silently assumed, in §21.

16. DEPLOYMENT / ENVIRONMENTS

Minimum required — not over-engineered.

Environment	Purpose	Database	Notes
Development	Local/shared dev work	Separate Atlas project/dev cluster (or local MongoDB)	Seed data only — never real PII
Staging	Pre-production verification, UAT	Separate Atlas cluster, mirrors production config	Sanitized/synthetic data; used to verify migrations before production
Production	Live system	Production Atlas cluster with backups + PITR	Real data; access restricted to deploy pipeline + on-call

Environment variables & secrets: database connection string, JWT/session signing secret, object-storage credentials, WhatsApp/telephony/email/payment-gateway/geocoding API keys, Redis connection string — one value set per environment, stored in a managed secrets store (§15), never committed to source. A `.env.example` enumerates required keys without values.

Database separation: each environment is a fully separate Atlas project/cluster — no shared database between dev/staging/production, so a staging bug can never touch production data.

Migrations: schema changes (new fields, new indexes) are additive and backward-compatible by default, given MongoDB’s flexible schema; any migration requiring a data backfill (e.g. computing `dedupe_status` on existing records, or the education historical-backfill decision in §21) ships as an idempotent, re-runnable migration script, run first in staging, then production, with a rollback plan documented per script.

Backups: daily automated Atlas backups with point-in-time recovery in staging and production (§15); development is not backed up.

Logging: structured application logs (request id, actor, route, latency, status) shipped to a centralized log store; `audit_logs` (§4/§15) is the durable business-event record, distinct from operational logging.

Monitoring: API latency/error-rate alerting against the performance targets implied by each screen’s data contract (Institution Map viewport queries, Global Search, dashboard load); MongoDB Atlas’s built-in cluster monitoring (CPU, connections, slow queries) for database health; scheduled-job (MoU expiry, notification dispatch, map cluster precomputation, daily metrics aggregation) success/failure alerting, since these are silent-failure-prone by nature.

Deployment: standard CI pipeline — build, test, deploy to staging on merge to a staging branch, manual promotion to production; no blue/green or canary complexity required at Phase 1 scale, but deploys should be zero-downtime (rolling) given the API layer is stateless.

Rollback: previous deployed build/image kept ready for immediate redeploy; database migrations are written to be forward-only where possible (additive), with any destructive migration requiring an explicit, reviewed rollback script before it ships.

17. IMPLEMENTATION ORDER

Dependency-derived, not navigation-derived — Institutions depend on Relationship/Activity existing first, and Education depends on Person existing first so a Student is never modeled as a standalone identity.

Phase	Builds	Dependencies	Exit criteria
1A — Foundation	Auth (login/logout/session/reset), Users, Roles/Permissions, App Shell + role-filtered Sidebar, MongoDB provisioning (dev/staging/prod), audit-log write-through plumbing	—	A user can log in, see a role-appropriate sidebar, and every subsequent write is captured in <code>audit_logs</code>
1B — Core CRM	Person (CRUD + dedup-on-create), Organization (incl. institution category + hierarchy), Relationship (CRUD + RelationshipPanel), Activity (logging +	1A	A Person and Organization can be created without duplicates, connected via a Relationship, and an Activity logged against both appears on each one’s

	polymorphic timeline + ActivityTimeline/Composer), Task, GlobalSearch (Atlas Search on Person/Organization), Document upload		timeline from a single write
1C — Commercial CRM	Lead (CRUD + list/filter), Lead→Opportunity conversion (dedup-safe), Opportunity (CRUD + PipelineBoard drag-to-stage), Notifications for lead assignment/untouched/stage-change	1B	A Lead converts to exactly one Opportunity with no duplicate Person/Organization, and the pipeline board reflects stage changes with notifications firing
1D — Institution Network	Institution data model (2dsphere index, district/type filters), Institution Map (viewport query, clustering job, Map/MapMarker/MapCluster), InstitutionDrawer + Institution 360 data contract, MoU CRUD + lifecycle + expiry scheduled job + renewal, MoUExpiring notification wiring	1B (Relationship/Activity), 1C (Opportunity, for the 360's opportunity tab)	The full Madurai → Map → Institution → 360 → MoU → Activity → Contact → Opportunity journey (\$10) works end-to-end
1E — Education Integration	StudentProfile on Person + Enrollment CRUD, Attendance + DailyProgress logging screens, Course/CareerPath catalog CRUD, rule-based Recommendation engine tied to Student	1B (Person must exist first)	A Student enrolls without creating a duplicate Person, attendance/progress log correctly, and course recommendations render from the weighted-factor rules
1F — Executive Dashboard & Reporting	<code>daily_metrics</code> aggregation job, Dashboard screen + KPI Cards + drill-through	1C, 1D, 1E must all be producing real data first	Dashboard KPIs (revenue by vertical, pipeline, win rate, collections, active MoUs, critical tasks, recent activity) load within the performance target (\$16 monitoring) against real Phase 1 data, with financial KPIs correctly hidden for non-privileged roles

The Executive Dashboard is sequenced last deliberately: it aggregates data every other phase produces, so building it earlier would mean building it against fixtures, then rebuilding it against real data once 1C/1D/1E ship.

18. DEVELOPER BACKLOG

Epic: Foundation (Phase 1A)

ID	Story	Type	Dependencies	Priority	Acceptance criteria
F-1	Auth: login/logout/session/reset	Full-stack	—	P0	User can log in with valid credentials, session persists per \$15 expiry rules, reset flow completes end-to-end
F-2	RBAC: role/permission model + middleware enforcement	Backend	F-1	P0	Every route in \$5 rejects a request lacking the required permission with 403
F-3	App Shell + Sidebar with role-filtered nav	Frontend	F-2	P0	Sidebar shows only nav items the logged-in role has <code>view</code> on
F-4	Audit log write-through on all governed collections	Backend	F-2	P1	Every write to a \$15-listed collection produces a matching <code>audit_logs</code> entry with correct before/after diff

Epic: Core CRM (Phase 1B)

ID	Story	Type	Dependencies	Priority	Acceptance criteria
C-1	Person CRUD + dedup check on create	Full-stack	F-2	P0	Creating a Person with an existing phone/email returns 409 with candidate match(es); no duplicate created
C-2	Organization CRUD incl. institution category + hierarchy	Full-stack	F-2	P0	Organization with <code>category=institution</code> exposes <code>institution_details</code> ; <code>parent_org_id</code> hierarchy renders correctly
C-3	Relationship CRUD + RelationshipPanel component	Full-stack	C-1, C-2	P0	A Relationship between any two entity types is creatable and visible on both entities' panels
C-4	Activity logging + polymorphic timeline query + ActivityTimeline/Composer	Full-stack	C-1, C-2	P0	One Activity with multiple <code>related_*_ids</code> appears on every referenced entity's timeline
C-5	Task creation from Activity <code>next_action</code> + TaskList	Full-stack	C-4	P1	Setting <code>next_action</code> on an Activity creates a linked Task visible in the assignee's TaskList
C-6	GlobalSearch across Person/Organization via Atlas Search	Full-stack	C-1, C-2	P1	Search-as-you-type returns grouped, badged results with exact and fuzzy matches
C-7	Document upload + attach-to-entity + FileUploader	Full-stack	C-1	P1	Uploaded file is retrievable only by users with <code>view</code> on the parent entity; re-upload versions correctly

Epic: Commercial CRM (Phase 1C)

ID	Story	Type	Dependencies	Priority	Acceptance criteria
S-1	Lead CRUD + list/filter	Full-stack	C-1, C-2	P0	Lead list filters by stage/owner/source/vertical correctly
S-2	Lead conversion flow (dedup-safe)	Full-stack	S-1	P0	Conversion produces exactly one Opportunity, no duplicate Person/Organization, Lead stamped as converted
S-3	Opportunity CRUD + PipelineBoard (drag-to-stage)	Full-stack	S-2	P0	Dragging a card between columns issues a <code>PATCH</code> and emits <code>OpportunityStageChanged</code>
S-4	Notification events for lead assignment/untouched/stage change	Backend	S-1, S-3, F-4	P1	Notification appears in <code>NotificationCenter</code> for each of the three triggers within the expected time window

Epic: Institution Network (Phase 1D)

ID	Story	Type	Dependencies	Priority	Acceptance criteria
I-1	Institution data model + 2dsphere index + district/type filters	Backend	C-2	P0	<code>GET /institutions</code> filters correctly on <code>district/type/management_type</code>

I-2	Institution Map: viewport query, clustering job, Map/MapMarker/MapCluster components	Full-stack	I-1	P0	Map loads centered on Madurai, pans/zooms within the \$16 performance target, clusters at zoomed-out levels
I-3	InstitutionDrawer + Institution 360 data contract endpoint	Full-stack	I-1, C-3, C-4	P0	Drawer opens on marker click without an extra call beyond the lightweight profile fetch; 360 renders every \$10 section
I-4	MoU CRUD + lifecycle transitions + expiry scheduled job	Full-stack	C-2	P0	MoU created at <code>status=proposed</code> ; scheduled job transitions to <code>expiring</code> / <code>expired</code> at the correct thresholds
I-5	MoUExpiring notification wiring	Backend	I-4, S-4	P1	Owner notified at 90/60/30/7 days with escalating priority

Epic: Education Integration (Phase 1E)

ID	Story	Type	Dependencies	Priority	Acceptance criteria
E-1	StudentProfile on Person + Enrollment CRUD	Full-stack	C-1	P0	Enrolling reuses an existing Person on phone/email match; never creates a duplicate identity
E-2	Attendance + DailyProgress logging screens	Full-stack	E-1	P0	Attendance/progress records reference <code>enrollment_id</code> (not embedded), per \$4
E-3	Course/CareerPath catalog CRUD	Full-stack	—	P1	Course list/detail CRUD works independent of enrollment volume
E-4	Recommendation engine (rule-based, non-AI) tied to Student	Full-stack	E-1, E-3	P2	Weighted-factor score renders per \$12's bands (≥85% Recommended / 40-84% Available / <40% Hidden)

Epic: Executive Dashboard (Phase 1F)

ID	Story	Type	Dependencies	Priority	Acceptance criteria
D-1	<code>daily_metrics</code> aggregation job	Backend	S-3, I-4, E-2	P1	Job runs nightly, produces one document per metric per day, matches live-query totals within tolerance
D-2	Dashboard screen + KPI Cards + drill-through	Full-stack	D-1	P1	Each KPI Card drill-through opens the correctly filtered list view; financial KPIs hidden per \$8 for non-privileged roles

19. ACCEPTANCE TESTS

Executable functional tests. Each is pass/fail against the running system.

- Person creation** — submitting a new Person with a phone/email that doesn't match an existing record creates it; a match blocks creation (`409`) and surfaces the existing record.
- Organization creation** — Organization created with a `category` ; `category=institution` exposes `institution_details` fields.
- Institution creation** — Institution (Organization subtype) created with `lat/lng` , `district` , `institution_type` correctly geo-indexed and immediately queryable via `GET /institutions/map` .
- Relationship creation** — a Relationship between any two entity types is creatable, appears on both entities' Relationship panels, and is time-bounded (`start_date` set, `end_date` nullable).
- Lead creation** — Lead requires a Person; creating with a new person's details creates that Person first (no separate identity created outside the flow).
- Lead conversion** — conversion produces exactly one Opportunity, no duplicate Person/Organization, and stamps the Lead `converted_to_opportunity_id` .
- Opportunity creation** — Opportunity requires an account and contact; appears on the pipeline board in its stage's column.
- Activity creation** — an Activity with multiple `related_*` ids appears on every one of those entities' timelines from a single write.
- Timeline display** — Person/Organization/Institution timelines are chronological, paginated, and filterable by type.
- Task creation** — an Activity's `next_action` produces a Task without re-entering context; Task appears in the assignee's TaskList.
- MoU creation** — MoU created against an Institution/Organization starts at `status=proposed` .
- MoU expiry** — the scheduled job transitions an MoU to `expiring` at 90/60/30/7 days out and to `expired` past `end_date` with no renewal, firing notifications at each threshold.
- Institution map** — loads centered on Madurai district by default; markers render within the \$16 performance target even at large institution counts.
- District filter** — map and list both narrow to the selected district using the same filter parameter.
- Institution filter** — relationship-status and MoU-status filters reflect live Relationship/MoU data, not a stale stored field.
- Institution selection** — marker click opens `InstitutionDrawer` with a profile summary before the full 360 loads.
- Institution 360** — every section in \$10's data contract renders, each independently/lazily loading.
- Institution timeline** — same polymorphic mechanism as Person/Organization timelines, filtered to that institution.
- Institution activity creation** — `ActivityComposer` opened from Institution 360 pre-fills `related_institution_id` .
- Education student creation** — enrolling a Student reuses an existing Person if phone/email matches; never creates a duplicate identity.
- Enrollment** — a `studied_at` Relationship links the Student's Person to the Institution/course provider.
- Learning progress** — Attendance and DailyProgress entries reference `enrollment_id` correctly and roll up into the Student/Trainer/Admin learning dashboards.
- Certification** — a certification outcome is recorded against the Enrollment/Person without creating a new identity.
- Placement** — placement creates a new `employed_by` Relationship on the same Person, plus an Opportunity/Activity record, while the prior `studied_at` Relationship remains intact as history.
- Alumni** — ending the `studied_at` relationship (setting `end_date`) and adding an `employed_by` relationship preserves both as history on the same Person.
- Cross-vertical relationship** — a Person with Alumni + Corporate Contact + Referral Source roles simultaneously shows all three on one Person 360 page, tied to three different Opportunities/Organizations.

CRITICAL TEST — identity continuity

A person must be able to move through **Prospect → Student → Alumni → Employee → Corporate Contact** without duplicate identity records. Verification: after each transition (enrollment, graduation, employment, becoming a corporate contact/decision maker for a deal), exactly one `persons` document exists for that individual, and their full history is the ordered set of their Relationship records — never a second Person row at any step.

CRITICAL TEST — institution journey

Madurai → Map → Institution → Institution 360 → MoU → Activity → Contact → Opportunity must work end-to-end: 1. Map loads on Madurai, an institution with no relationship is visible and selectable. 2. Selecting it opens the Drawer, then the full Institution 360. 3. A Contact (e.g. Placement Officer) is created as a Person with a role-bearing Relationship to the institution. 4. An Activity is logged referencing the employee (actor), the Contact (`related_person_id`), and the Institution (`related_institution_id`) — it appears on all three timelines from the single write. 5. An MoU is created against the Institution and appears in its 360 MoU tab. 6. An Opportunity is created against the Institution, linked to the Contact and the Activity. 7. Every step above is independently verifiable via its own acceptance test (3–19) and the whole chain completes without any duplicate Person/Organization record.

20. PHASE 1 DEFINITION OF DONE

Testable statements — every item below must be independently verifiable against the running system, not asserted.

- Users can log in, and their session expires per the 8-hour sliding / 30-day idle rule (\$15).
- Users can create a Person, and the system blocks a duplicate on exact phone/email match, returning the existing candidate.
- Users can create an Organization, including the Institution subtype with geospatial and academic fields.

- Users can create a Relationship between any two of Person/Organization/Institution, and it is time-bounded and visible from both sides.
- Users can log an Activity against up to five related entities in a single write, and it appears on every one of those entities’ timelines.
- Users can create a Lead and convert it to an Opportunity without creating a duplicate Person or Organization.
- Users can drag an Opportunity between pipeline stages on the Kanban board, and the change is persisted and triggers the correct notification.
- The Institution Map loads centered on Madurai district by default and supports district/type/management/relationship/MoU/recency/priority filtering.
- Selecting an institution marker opens a Drawer, and “Open Institution 360” loads the full data contract specified in §10.
- Users can create an MoU against an Institution/Organization, and the system automatically transitions its status to `expiring / expired` on the correct schedule, firing notifications at 90/60/30/7 days.
- Users can enroll a Student, and the system reuses the existing Person record if one matches, never creating a duplicate identity.
- Attendance and Daily Progress entries are recorded per-Enrollment and roll up correctly into the Student/Trainer/Admin learning dashboards.
- The Executive Dashboard renders revenue-by-vertical, pipeline, win rate, collections, active-MoU KPIs, critical tasks, and recent activity, scoped to the requesting user’s filters.
- Unauthorized roles cannot view financial KPIs, payment data, or MoU/opportunity records outside their permission grant (§8) — verified by attempting each restricted action as each non-privileged role and confirming a `403`.
- Every write to a governed collection (§15) produces a corresponding, immutable `audit_logs` entry.
- API returns `409` with candidate matches on a duplicate Person creation attempt, and `403` on any permission-denied request, consistently across every route in §5.
- A single Person record can carry Student, Alumni, Employee, and Corporate Contact roles simultaneously, each as a distinct Relationship, with no duplicate Person row at any point (validated by the critical acceptance tests in §19).
- No AI scoring, recommendation model, or automated inference runs anywhere in the shipped system — every “AI Insight” surfaced in the UI is a labeled, rule-based computation, not a live model (§1, §16 boundary).
- Notifications for every trigger listed in §13 fire to the correct recipient via the correct channel within the expected timing window.
- All Phase 1 screens listed in §6 are reachable, permission-gated correctly, and render their specified loading/empty/error states.

21. UNRESOLVED DECISIONS

Only genuine gaps — decisions neither source document resolves.

#	Decision	Why required	Options	Recommended option	Who should approve
1	Fuzzy person deduplication tolerance (beyond exact phone/email match)	Exact-match dedup will miss same-person records entered with typo’d names or reformatted phone numbers; fuzzy matching is valuable but tuning it wrong causes false-merge risk	(a) Ship Phase 1 with exact-match only, review merge-queue volume post-launch; (b) build fuzzy matching (name-similarity, phonetic phone/name) now	(a) — ship exact-match only in Phase 1; revisit only if the sales/counsellor team reports duplicate volume that makes it urgent	Founder/Product
2	Financial visibility (<code>F</code>) and Delete (<code>D</code>) grants per role in the RBAC matrix (§8)	Both source documents flag this as the highest-business-risk pair of permission columns and neither fully confirms every cell	Lock the Phase 1 default matrix as drafted; or review/adjust before build	Review before lock — the drafted matrix is a defensible default, not a confirmed business rule	Founder
3	Authoritative institution directory source for Tamil Nadu/Madurai data	The Institution Map needs real institution data at launch; no unverified data should be seeded	License/scrape an authoritative source (e.g. state education department listings, AISHE/UDISE); or manual data entry for an initial subset	Source an authoritative directory before 1D ships; manual entry only as a bridge for a small initial set	Product/Operations
4	Historical education-record backfill scope into Person/Relationship	Migrating years of existing education data into the universal model is a separate effort from building the model itself	(a) Backfill only current/active students and staff at 1E ship; (b) full historical backfill immediately; (c) backfill as a later background job	(a) at 1E ship, with (c) as a follow-on background migration	Engineering lead + Product
5	Data retention window and deletion-on-request policy	The schema supports soft-delete/hard-delete structurally, but the retention period itself is a compliance decision, not a technical one	Define per Kaizen’s DPDP (or equivalent) compliance posture	Not resolvable here — requires legal/compliance input	Legal/Compliance + Founder
6	MFA requirement for Phase 1 login	Neither source mandates MFA, but security posture may require it before go-live for certain organizations	(a) Ship Phase 1 without MFA, add Phase 2; (b) require MFA Day 1	(a) unless the organization’s security policy requires it Day 1 — confirm before build, don’t assume	Founder/Security stakeholder
7	Configurable notification thresholds (lead-untouched days, at-risk threshold, institution-inactive days, proposal-pending days)	Both sources reference these as “configurable” without specifying default values or where they’re configured in Phase 1	(a) Hardcode sensible defaults via environment variables for Phase 1, expose a Settings UI later; (b) build a Settings UI now	(a) — environment-variable defaults now, Settings UI is a Phase 2 item consistent with §1’s “Settings & role/permission management UI” deferral	Product